University of Botswana, Department of Computer Science

CSI142 Object Oriented Programming (Semester II 2025/26)

Week 4 Lab: Classes, Constructors, Encapsulation, UML

Week 4 Lab, Tuesday 24 February 2026

Learning outcomes for this lab

- Implement Java classes with private fields, constructors, and methods.
- Apply encapsulation: fields are private, state changes happen through methods.
- Use constructors and simple validation to keep objects in a valid state.
- Create and use objects in a small application, including an ArrayList of objects.
- Produce a basic UML class diagram with visibility (+ public, - private).

What you will submit

- Student.java
- BankAccount.java
- App.java
- A UML diagram for Student and BankAccount (PDF or image). If you cannot create a diagram file, include an ASCII UML diagram in a text file.

Follow the course policy on academic integrity. Submit your own work unless group work is explicitly allowed.

Starter code

Download the Lab 4 starter code zip from Moodle. Open the folder in VS Code and run:

```
javac *.java
```

```
java App
```

If your files are in a src folder, compile from that folder or adjust commands accordingly.

Task A: Implement the Student class (core)

Create a Student class that models a university student.

Requirements

- Fields (private): id (String), name (String), major (String), gpa (double).
- Constructor: Student(String id, String name, String major, double gpa).
- Validation: gpa must be between 0.0 and 4.0. If invalid, set to 0.0 or clamp into range (choose one approach and be consistent).
- Methods (public): getId(), getName(), getMajor(), getGpa().
- Method (public): changeMajor(String newMajor). Do not accept blank majors.
- Method (public): updateGpa(double newGpa) with the same validation rule as the constructor.

- Optional but recommended: toString() for readable printing.

Task B: Implement the BankAccount class (core)

Create a BankAccount class that protects account balance using encapsulation.

Requirements

- Fields (private): accountNumber (String), ownerName (String), balance (double).
- Constructors:
- 1) BankAccount(String accountNumber, String ownerName) sets balance to 0.0
- 2) BankAccount(String accountNumber, String ownerName, double initialBalance) with validation (no negative starting balance).
- Methods (public): getBalance().
- Method (public): deposit(double amount). Only accept positive deposits.
- Method (public): withdraw(double amount) returns boolean. Only allow withdrawal if amount is positive and amount <= balance.
- Optional but recommended: toString() for readable printing.

Task C: Integrate and test in App.java

Complete App.java so that it demonstrates your classes.

- Create at least 2 Student objects and print their details.
- Store Student objects in an ArrayList<Student> and loop through it.
- Create at least 1 BankAccount and perform a deposit and a withdrawal.
- Print the balance and the result of the withdrawal (true or false).

Task D: UML diagram

Create a UML class diagram for Student and BankAccount.

- Include class name, fields, and methods.
- Include visibility: - for private fields, + for public methods.
- Include parameter types and return types for methods.

Extension (bonus)

If you finish early, extend App.java into a small menu-driven program:

- Use a do-while loop and a switch statement to offer choices: deposit, withdraw, show balance, exit.
- Keep input handling in App.java, keep account rules inside BankAccount.

Marking guide (summary)

Your lab will be graded out of 10 marks. The checklist below shows a typical breakdown.

Area	What we look for	Marks (example)
Student class	Private fields, constructor, getters, changeMajor, updateGpa, validation	4
BankAccount class	Private fields, constructors, getBalance, deposit/withdraw rules	4
App integration	Creates and tests objects, uses ArrayList<Student>, clear output	2

Bonus: menu extension and/or toString implementation may earn extra credit allows.

Troubleshooting tips

- If you see 'cannot find symbol', check spelling and method names.
- If you see 'variable might not have been initialized', make sure your constructor sets every field.
- If you get ArrayList errors, confirm you imported java.util.ArrayList and used ArrayList<Student> correctly.
- Compile often. Fix one error at a time.